



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ
CENTRO REGIONAL DE COCLÉ
FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES
LICENCIATURA EN CIBERSEGURIDAD

Base de Datos Seguras

Practica #9

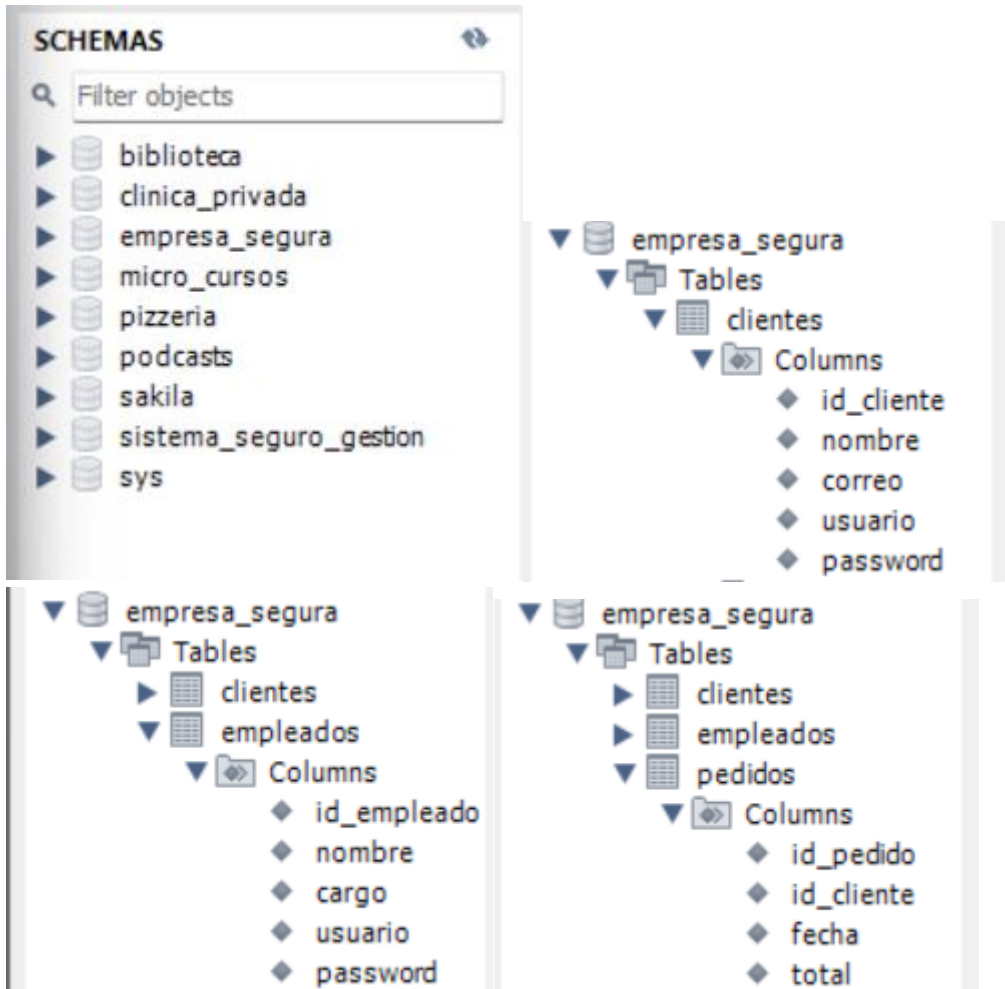
ESTUDIANTE:
JOHNNY RIVAS

PROFESOR:
Antonio Caballero

Laboratorio: SQL Injection en MySQL Server

Ataques, Impacto y Mitigación

Parte 1 - Crear la Base de Datos



Parte 2 - Insertar Información

```

1 • INSERT INTO clientes(nombre,correo,usuario,password)
2 VALUES
3 ('Juan Perez','juan@empresa.com','juan','12345'),
4 ('Maria Lopez','maria@empresa.com','maria','abc123'),
5 ('Carlos Diaz','carlos@empresa.com','carlos','admin123'),
6 ('Ana Ruiz','ana@empresa.com','anita','clave123'),
7 ('Pedro Soto','pedro@empresa.com','pedro','123456');

```

Output :

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 3	15:34:27	USE empresa_segura	0 row(s) affected	0.000 sec
✓ 4	15:36:02	CREATE TABLE clientes(id_cliente INT AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(50), correo VARCHAR(100), usuario VARCHAR(50), password VARCHAR(50));	0 row(s) affected	0.031 sec
✓ 5	15:36:36	CREATE TABLE empleados(id_empleado INT AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(50), correo VARCHAR(100), usuario VARCHAR(50), password VARCHAR(50));	0 row(s) affected	0.015 sec
✓ 6	15:37:33	CREATE TABLE pedidos(id_pedido INT AUTO_INCREMENT PRIMARY KEY, id_cliente INT, id_empleado INT, fecha DATE, monto DECIMAL(10,2));	0 row(s) affected	0.016 sec
✓ 7	15:39:06	INSERT INTO clientes(nombre,correo,usuario,password) VALUES ('Juan Perez','juan@empresa.com','juan','12345'),('Maria Lopez','maria@empresa.com','maria','abc123'),('Carlos Diaz','carlos@empresa.com','carlos','admin123'),('Ana Ruiz','ana@empresa.com','anita','clave123'),('Pedro Soto','pedro@empresa.com','pedro','123456');	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.000 sec

```

1 • INSERT INTO empleados(nombre,cargo,usuario,password)
2 VALUES
3 ('Luis Torres','Administrador','admin','admin123'),
4 ('Rosa Vega','Gerente','rosa','rosa123'),
5 ('Mario Diaz','Ventas','mario','ventas123');

```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 4	15:36:02	CREATE TABLE clientes(id_cliente INT AUTO_IN...	0 row(s) affected	0.031 sec
✓ 5	15:36:36	CREATE TABLE empleados(id_empleado INT AU...	0 row(s) affected	0.015 sec
✓ 6	15:37:33	CREATE TABLE pedidos(id_pedido INT AUTO_I...	0 row(s) affected	0.016 sec
✓ 7	15:39:06	INSERT INTO clientes(nombre,correo,usuario,pass...	5 row(s) affected Records: 5 Duplicates: 0 Wamin...	0.000 sec
✓ 8	15:39:38	INSERT INTO empleados(nombre,cargo,usuario.p...	3 row(s) affected Records: 3 Duplicates: 0 Wamin...	0.000 sec

```

1 • INSERT INTO pedidos(id_cliente,fecha,total)
2 VALUES
3 (1, '2026-06-01',250),
4 (1, '2026-06-10',120),
5 (2, '2026-06-05',480),
6 (3, '2026-06-08',310),
7 (4, '2026-06-15',700),
8 (5, '2026-06-20',950);

```

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
✓ 5	15:36:36	CREATE TABLE empleados(id_empleado INT AU...	0 row(s) affected	0.015 sec
✓ 6	15:37:33	CREATE TABLE pedidos(id_pedido INT AUTO_I...	0 row(s) affected	0.016 sec
✓ 7	15:39:06	INSERT INTO clientes(nombre,correo,usuario,pass...	5 row(s) affected Records: 5 Duplicates: 0 Wamin...	0.000 sec
✓ 8	15:39:38	INSERT INTO empleados(nombre,cargo,usuario.p...	3 row(s) affected Records: 3 Duplicates: 0 Wamin...	0.000 sec
✓ 9	15:40:06	INSERT INTO pedidos(id_cliente,fecha,total) VAL...	6 row(s) affected Records: 6 Duplicates: 0 Wamin...	0.015 sec

Parte 3 - Consultas Normales

	id_cliente	nombre	correo	usuario	password
▶	1	Juan Perez	juan@empresa.com	juan	12345
	2	Maria Lopez	maria@empresa.com	maria	abc123
	3	Carlos Diaz	carlos@empresa.com	carlos	admin123
	4	Ana Ruiz	ana@empresa.com	anita	clave123
	5	Pedro Soto	pedro@empresa.com	pedro	123456

	id_empleado	nombre	cargo	usuario	password
▶	1	Luis Torres	Administrador	admin	admin123
	2	Rosa Vega	Gerente	rosa	rosa123
	3	Mario Diaz	Ventas	mario	ventas123

	id_pedido	id_cliente	fecha	total
▶	1	1	2026-06-01	250.00
	2	1	2026-06-10	120.00
	3	2	2026-06-05	480.00
	4	3	2026-06-08	310.00
	5	4	2026-06-15	700.00
	6	5	2026-06-20	950.00

Parte 4 - Simulación del Sistema Web Vulnerable

	id_empleado	nombre	cargo	usuario	password
▶	1	Luis Torres	Administrador	admin	admin123

Parte 5 - SQL Injection

Ataque 1: Bypass de login

	id_empleado	nombre	cargo	usuario	password
▶	1	Luis Torres	Administrador	admin	admin123
	2	Rosa Vega	Gerente	rosa	rosa123
	3	Mario Diaz	Ventas	mario	ventas123

Ataque 2: Inyección en usuario y contraseña

	id_empleado	nombre	cargo	usuario	password
▶	1	Luis Torres	Administrador	admin	admin123
	2	Rosa Vega	Gerente	rosa	rosa123
	3	Mario Diaz	Ventas	mario	ventas123

Ataque 3: Inyección en búsqueda de clientes

	id_cliente	nombre	correo	usuario	password
▶	1	Juan Perez	juan@empresa.com	juan	12345

	id_cliente	nombre	correo	usuario	password
▶	1	Juan Perez	juan@empresa.com	juan	12345
	2	Maria Lopez	maria@empresa.com	maria	abc123
	3	Carlos Diaz	carlos@empresa.com	carlos	admin123
	4	Ana Ruiz	ana@empresa.com	anita	clave123
	5	Pedro Soto	pedro@empresa.com	pedro	123456

Ataque 4: Extracción de información con UNION

	id_empleado	nombre	cargo	usuario	password
▶	1	Luis Torres	Administrador	admin	admin123

	Juan Perez	juan@empresa.com	juan	12345	1
	Maria Lopez	maria@empresa.com	maria	abc123	2
	Carlos Diaz	carlos@empresa.com	carlos	admin123	3
	Ana Ruiz	ana@empresa.com	anita	clave123	4
	Pedro Soto	pedro@empresa.com	pedro	123456	5

Ataque 5: Eliminación de información (DELETE)

	id_cliente	nombre	correo	usuario	password
--	------------	--------	--------	---------	----------

Ataque 6: Actualización Masiva (UPDATE)

```

1 • UPDATE clientes
2   SET password='hackeado'
3   WHERE usuario='juan'
4   OR '1'='1';

```

Output :

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 24	15:59:07	DELETE FROM pedidos WHERE id_cliente IN (S...	6 row(s) affected	0.000 sec
✓ 25	15:59:12	DELETE FROM clientes WHERE usuario='juan' ...	5 row(s) affected	0.000 sec
✓ 26	15:59:59	SELECT * FROM clientes LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
✓ 27	16:00:37	INSERT INTO clientes(nombre,correo,usuario,pas...	5 row(s) affected Records: 5 Duplicates: 0 Wami...	0.016 sec
✓ 28	16:00:57	UPDATE clientes SET password='hackeado' WH...	5 row(s) affected Rows matched: 5 Changed: 5 ...	0.016 sec

	id_cliente	nombre	correo	usuario	password
▶	6	Juan Perez	juan@empresa.com	juan	hackeado

Ataque 7: Comentario SQL (--)

```
1 • SELECT *
2 FROM clientes
3 WHERE nombre='Juan Perez'
4 -- ';
```

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	id_cliente	nombre	correo	usuario	password
▶	6	Juan Perez	juan@empresa.com	juan	hackeado

Parte 6 - Evaluación del Impacto

¿Qué tablas fueron afectadas?

Fueron afectadas las tablas empleadas (Ataques 1 y 2) y clientes (Ataques 3, 4, 5, 6 y 7).

¿Cuántos registros fueron comprometidos?

Los tres registros de la tabla empleados quedaron expuestos en los Ataques 1 y 2, y los cinco registros de la tabla clientes se vieron comprometidos en los Ataques 3, 5 y 6 (consultados, eliminados y modificados respectivamente).

¿Qué datos confidenciales fueron expuestos?

Se expusieron nombres de usuario, contraseñas y correos electrónicos tanto de empleados como de clientes, además de datos personales como nombres completos.

¿Qué operaciones críticas realizó el atacante?

El atacante realizó bypass de autenticación sin conocer la contraseña real, extracción de información combinando tablas mediante UNION, eliminación masiva de registros con DELETE, modificación masiva de contraseñas con UPDATE, y evasión de la lógica de la consulta original mediante el comentario --.

¿Qué controles de seguridad fallaron?

Fallaron la validación de entrada del usuario, el uso de consultas concatenadas en lugar de consultas preparadas, el almacenamiento de contraseñas en texto plano, y el uso de un usuario de base de datos con privilegios excesivos para la aplicación.

Parte 7 - Técnicas de Mitigación

```
1 • GRANT SELECT,INSERT,UPDATE
2 ON empresa_segura.*
3 TO 'app_empresa';
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 30	16:02:40	SELECT * FROM clientes WHERE nombre='Juan...	1 row(s) returned	0.000 sec / 0.000 sec
✓ 31	16:03:01	SELECT * FROM clientes WHERE nombre='Juan...	1 row(s) returned	0.000 sec / 0.000 sec
✓ 32	16:03:45	SELECT * FROM clientes WHERE nombre='Juan...	1 row(s) returned	0.015 sec / 0.000 sec
✓ 33	16:16:10	CREATE USER 'app_empresa' IDENTIFIED BY '...	0 row(s) affected	0.031 sec
✓ 34	16:16:38	GRANT SELECT,INSERT,UPDATE ON empresa...	0 row(s) affected	0.000 sec

```

1 • INSERT INTO clientes
2   (nombre,correo,usuario,password)
3   VALUES
4   ('Pedro',
5    'pedro@empresa.com',
6    'pedro',
7    SHA2('12345',256));

```

Output :

📄 Action Output ▾

#	Time	Action	Message	Duration / Fetch
✓ 31	16:03:01	SELECT * FROM clientes WHERE nombre='Juan...	1 row(s) returned	0.000 sec / 0.000 sec
✓ 32	16:03:45	SELECT * FROM clientes WHERE nombre='Juan...	1 row(s) returned	0.015 sec / 0.000 sec
✓ 33	16:16:10	CREATE USER 'app_empresa' IDENTIFIED BY '...	0 row(s) affected	0.031 sec
✓ 34	16:16:38	GRANT SELECT,INSERT,UPDATE ON empresa...	0 row(s) affected	0.000 sec
✓ 35	16:17:55	INSERT INTO clientes (nombre,correo,usuario,pa...	1 row(s) affected	0.000 sec

```

!
"
--
#
;
UNION
DROP

```

Parte 8 - Verificación

```
1 • SELECT *
2 FROM empleados
3 WHERE usuario='admin'
4 AND password='' OR '1'='1';
```

Result Grid					
		Filter Rows:		Export:	Wrap Cell Content:
	id_empleado	nombre	cargo	usuario	password
▶	1	Luis Torres	Administrador	admin	admin123
	2	Rosa Vega	Gerente	rosa	rosa123
	3	Mario Diaz	Ventas	mario	ventas123

Ventas

```

1 • SELECT *
2 FROM clientes
3 WHERE nombre=''
4 OR '1'='1';

```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 					
	id_cliente	nombre	correo	usuario	password
▶	6	Juan Perez	juan@empresa.com	juan	hackeado
	7	Maria Lopez	maria@empresa.com	maria	hackeado
	8	Carlos Diaz	carlos@empresa.com	carlos	hackeado
	9	Ana Ruiz	ana@empresa.com	anita	hackeado
	10	Pedro Soto	pedro@empresa.com	pedro	hackeado
	11	Pedro	pedro@empresa.com	pedro	5994471abb01112afcc18159f6cc74b4f511b998...

¿Funcionó?

No, la inyección ya no permite el bypass ni la extracción de datos como antes.

¿Por qué ya no funciona?

Porque la consulta ya no concatena directamente el texto ingresado por el usuario dentro del comando SQL, sino que utiliza parámetros (consultas preparadas), por lo que el texto inyectado se trata como un simple valor de búsqueda y no como parte del código SQL.

¿Qué mecanismo evitó el ataque?

El uso de consultas preparadas junto con la validación de entrada y el usuario de aplicación con privilegios mínimos.

Preguntas de Análisis

¿Qué es SQL Injection?

Es una vulnerabilidad que permite a un atacante insertar o manipular sentencias SQL dentro de una consulta, aprovechando que la aplicación construye la consulta concatenando directamente el texto ingresado por el usuario.

¿Por qué ocurre?

Ocurre porque la aplicación no valida ni separa correctamente los datos ingresados por el usuario del código SQL que se ejecuta, permitiendo que ese texto altere la lógica de la consulta original.

¿Cuál fue el ataque más peligroso?

El Ataque 5 (DELETE masivo), ya que eliminó de forma irreversible todos los registros de la tabla clientes, representando una pérdida total de información.

¿Qué información obtuvo el atacante?

Obtuvo nombres de usuario, contraseñas, correos electrónicos y nombres completos tanto de empleados como de clientes.

¿Qué impacto tendría esto en una empresa real?

Implicaría pérdida de confidencialidad de los datos de clientes y empleados, pérdida económica por fraude o robo de identidad, pérdida de disponibilidad de la información por eliminación de registros, daño reputacional y posibles sanciones legales por incumplimiento de protección de datos.

¿Qué técnicas de mitigación implementó?

Se implementó el principio del menor privilegio mediante un usuario de aplicación con permisos limitados, cifrado de contraseñas con SHA2, validación de datos de entrada, consultas preparadas, uso de roles, auditoría y respaldo periódico de la información.

¿Qué otros ataques similares conocen?

Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), Command Injection y ataques de fuerza bruta contra formularios de autenticación.

¿Qué papel desempeña el principio del menor privilegio?

Limita el daño que puede causar un ataque exitoso, ya que aunque el atacante logre inyectar código, el usuario de la base de datos no tendrá permisos para ejecutar operaciones críticas como eliminar tablas o crear nuevos usuarios.

¿Qué diferencia existe entre validar datos y usar consultas preparadas?

La validación de datos filtra o rechaza caracteres y patrones sospechosos antes de que lleguen a la consulta, mientras que las consultas preparadas separan estructuralmente el código SQL de los datos, de modo que el valor ingresado nunca se interpreta como parte del comando, sin importar qué caracteres contenga.

Si usted fuera el auditor de la empresa, ¿qué recomendaciones adicionales implementaría?

Recomendaría implementar un firewall de aplicaciones web (WAF), realizar pruebas de penetración periódicas, cifrar la conexión entre la aplicación y la base de datos, establecer políticas de contraseñas robustas, capacitar al equipo de desarrollo en prácticas de codificación segura, y establecer un plan de respuesta ante incidentes.